

Perguntas finais

Onde foi usada a fila?

A fila foi usada no atendimento normal das ocorrências, seguindo a ordem de chegada, ou seja, o comportamento FIFO. No arquivo `main.py`, a função `inserir_fila(ocorrencia)` chama `gerenciador_fluxo.adicionar_ocorrencia(ocorrencia)`, que envia a ocorrência para o fluxo geral do sistema. Já o atendimento acontece na função `atender_fila()`, que utiliza `gerenciador_fluxo.fila_chegada.dequeue()` para remover e atender a ocorrência mais antiga da fila. No README, essa estrutura aparece associada ao arquivo `fluxo.py`, com os métodos `FilaAtendimento.enqueue(ocorrencia)`, `FilaAtendimento.dequeue()` e `GerenciadorFluxo.adicionar_ocorrencia(ocorrencia)`.

Onde foi usada a pilha?

A pilha foi usada no histórico de ações do sistema, permitindo visualizar e desfazer a última ação realizada. Ela foi implementada no arquivo `main.py` por meio do vetor `historico_pilha` e da variável `topo`. Os principais métodos utilizados foram `adicionar_ao_historico(acao)`, que insere uma nova ação no topo da pilha, `exibir_historico()`, que mostra as ações da mais recente para a mais antiga, e `desfazer_ultima_acao()`, que remove a última ação registrada. Essa estrutura segue o comportamento LIFO, no qual o último elemento inserido é o primeiro a ser removido.

Onde foi usada a árvore?

Nos arquivos atuais, não foi implementada uma árvore separada, como uma árvore binária de busca. O conceito de árvore aparece de forma indireta na estrutura de heap, pois a heap é uma árvore binária completa representada internamente por um vetor. Isso aparece no arquivo `heap_prioridade.py`, principalmente nos métodos `subir(indice)` e `descer(indice)`, que calculam posições de pai e filhos usando fórmulas como $(indice - 1) // 2$, $2 * indice + 1$ e $2 * indice + 2$. Portanto, a árvore não foi usada diretamente como estrutura independente, mas sua lógica foi aplicada dentro da heap.

Onde foi usada a heap?

A heap foi usada para o atendimento por prioridade. Ela está implementada no arquivo `heap_prioridade.py`, na classe `HeapPrioridade`. Quando uma ocorrência é cadastrada no `main.py`, ela é inserida na heap pelo método `heap_prioridade.inserir(ocorrencia)`. Para atender a ocorrência mais urgente, a função `atender_prioridade()` chama `heap_prioridade.remover_maior_prioridade()`. Dentro da heap, os métodos mais importantes são `tem_maior_prioridade(ocorrencia_a, ocorrencia_b)`, `inserir(ocorrencia)`, `remover_maior_prioridade()`, `subir(indice)` e `descer(indice)`. A prioridade maior é atendida primeiro, e em caso de empate é usada a `ordem_chegada` como critério de desempate.

Onde foi usada a hash table?

A hash table foi usada para a busca rápida de ocorrências por nome do solicitante ou por tipo de ocorrência. Ela está implementada no arquivo `indice_hash.py`. A classe `NoHash` representa os nós usados no tratamento de colisões por encadeamento. A função `funcao_hash(chave)` transforma a chave textual em um índice numérico da tabela. A função `inserir_indice(chave, ocorrencia)` insere a ocorrência na posição correta da tabela. A função `indexar_ocorrencia(ocorrencia)` cadastra a mesma ocorrência duas vezes no índice: uma pelo campo `nome` e outra pelo campo `tipo`. A busca é feita por `buscar_no_indice(chave)`, chamada pela função `buscar_hash(chave)`. No `main.py`, a busca aparece na função `buscar_por_hash()`, que permite pesquisar por nome ou tipo.

Qual algoritmo de ordenação foi implementado?

Foi implementado o algoritmo Bubble Sort, no arquivo `ordenacao.py`. A função principal é `_bubble_sort(lista_geral, criterio, decrescente=False)`, que percorre a lista comparando elementos vizinhos e trocando suas posições quando necessário. Esse algoritmo é reutilizado pelos métodos `ordenar_por_id(lista_geral)`, `ordenar_por_nome(lista_geral)` e `ordenar_por_prioridade(lista_geral)`. No arquivo `main.py`, a função `ordenar_lista_manual()` chama esses métodos conforme a opção escolhida pelo usuário.

Qual estrutura foi mais adequada para busca rápida?

A estrutura mais adequada para busca rápida foi a hash table, implementada no arquivo `indice_hash.py`. Ela foi escolhida porque permite localizar ocorrências diretamente a partir de uma chave, como nome ou tipo, sem precisar percorrer toda a lista geral. Os principais métodos envolvidos nessa busca são `funcao_hash(chave)`, `buscar_no_indice(chave)` e `buscar_hash(chave)`.

Qual estrutura foi mais adequada para atendimento por prioridade?

A estrutura mais adequada para atendimento por prioridade foi a heap, implementada no arquivo `heap_prioridade.py`. Ela foi adequada porque mantém a ocorrência de maior prioridade na raiz da estrutura, facilitando o atendimento da ocorrência mais urgente. Os métodos principais usados para isso foram `inserir(ocorrencia)`, `remover_maior_prioridade()`, `subir(indice)`, `descer(indice)` e `tem_maior_prioridade(ocorrencia_a, ocorrencia_b)`.

Qual foi a maior dificuldade do grupo?

A maior dificuldade do grupo foi integrar várias estruturas de dados diferentes no mesmo sistema, mantendo os dados consistentes entre elas. Ao cadastrar uma ocorrência no `main.py`, ela precisava ser enviada para a fila, para a heap de prioridade, para a hash table e também para a lista geral do fluxo. Isso exigiu cuidado para que a mesma ocorrência pudesse ser listada, buscada, ordenada e atendida corretamente, sem duplicar informações ou perder o status atualizado. Outra dificuldade foi entender qual estrutura deveria ser usada para cada necessidade: fila para ordem de chegada, pilha para histórico, hash table para busca rápida, heap para prioridade e Bubble Sort para ordenação manual.